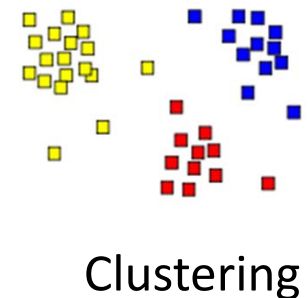
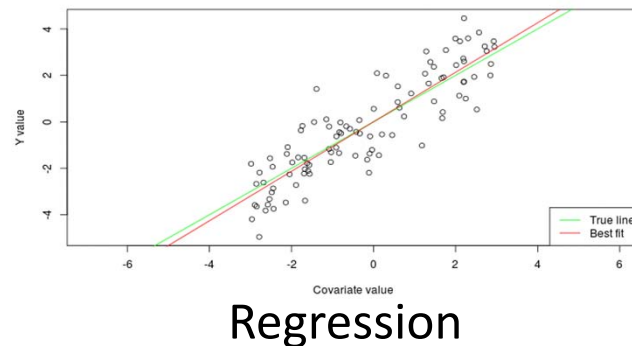
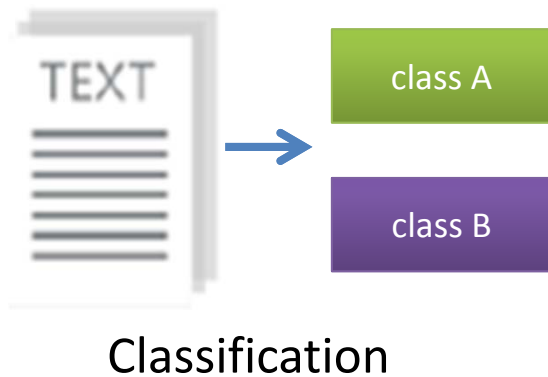


Machine Learning Types

Machine Learning Basics

- **Supervised**: learning with **labeled data**
 - Example: email classification, image classification
 - Example: regression for predicting real-valued outputs
- **Unsupervised**: discover patterns in **unlabeled data**
 - Example: cluster similar data points
- **Reinforcement learning**: learn to act based on **feedback/reward**
 - Example: learn to play Go



Supervised Learning

Machine Learning Basics

- **Supervised learning** categories and techniques
 - **Numerical classifier functions**
 - Linear classifier, perceptron, logistic regression, support vector machines (SVM), neural networks
 - **Parametric (probabilistic) functions**
 - Naïve Bayes, Gaussian discriminant analysis (GDA), hidden Markov models (HMM), probabilistic graphical models
 - **Non-parametric (instance-based) functions**
 - k -nearest neighbors, kernel regression, kernel density estimation, local regression
 - **Symbolic functions**
 - Decision trees, classification and regression trees (CART)
 - **Aggregation (ensemble) learning**
 - Bagging, boosting (Adaboost), random forest

Unsupervised Learning

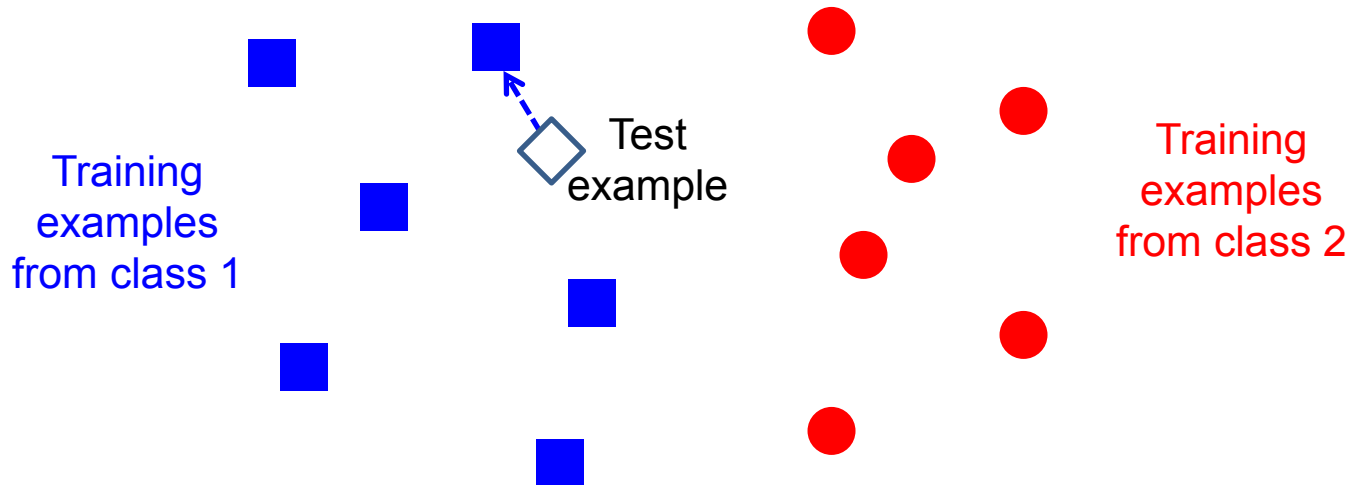
Machine Learning Basics

- *Unsupervised learning* categories and techniques
 - **Clustering**
 - k -means clustering
 - Mean-shift clustering
 - Spectral clustering
 - **Density estimation**
 - Gaussian mixture model (GMM)
 - Graphical models
 - **Dimensionality reduction**
 - Principal component analysis (PCA)
 - Factor analysis

Nearest Neighbor Classifier

Machine Learning Basics

- **Nearest Neighbor** – for each test data point, assign the class label of the nearest training data point
 - Adopt a distance function to find the nearest neighbor
 - Calculate the distance to each data point in the training set, and assign the class of the nearest data point (minimum distance)
 - It does not require learning a set of weights



Nearest Neighbor Classifier

Machine Learning Basics

- For image classification, the distance between all pixels is calculated (e.g., using ℓ_1 norm, or ℓ_2 norm)
 - Accuracy on CIFAR-10: 38.6%
- Disadvantages:
 - The classifier must remember all training data and store it for future comparisons with the test data
 - Classifying a test image is expensive since it requires a comparison to all training images

test image					training image					pixel-wise absolute value differences			
56	32	10	18		10	20	24	17		46	12	14	1
90	23	128	133		8	10	89	100		82	13	39	33
24	26	178	200	-	12	16	178	170	=	12	10	0	30
2	0	255	220		4	32	233	112		2	32	22	108

→ 456

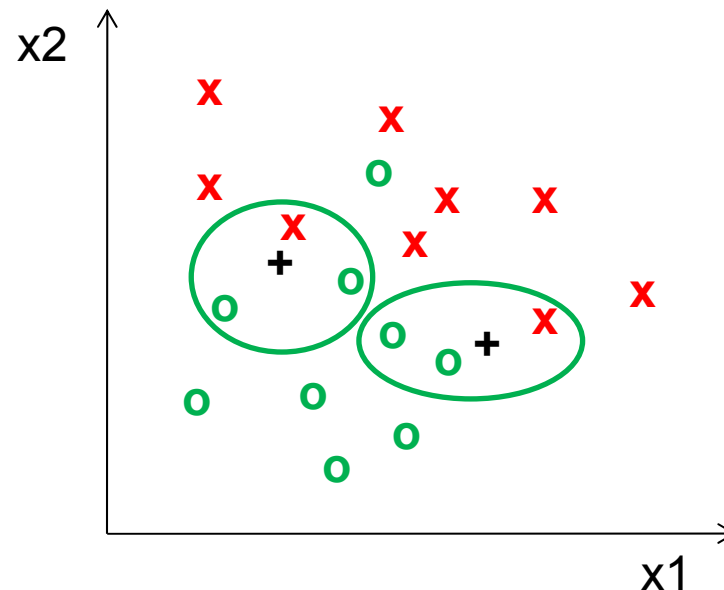
ℓ_1 norm
(Manhattan distance)

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

k -Nearest Neighbors Classifier

Machine Learning Basics

- **k -Nearest Neighbors** approach considers multiple neighboring data points to classify a test data point
 - E.g., 3-nearest neighbors
 - The test example in the figure is the + mark
 - The class of the test example is obtained by voting (based on the distance to the 3 closest points)



Linear Classifier

Machine Learning Basics

- *Linear classifier*

- Find a linear function f of the inputs x_i that separates the classes

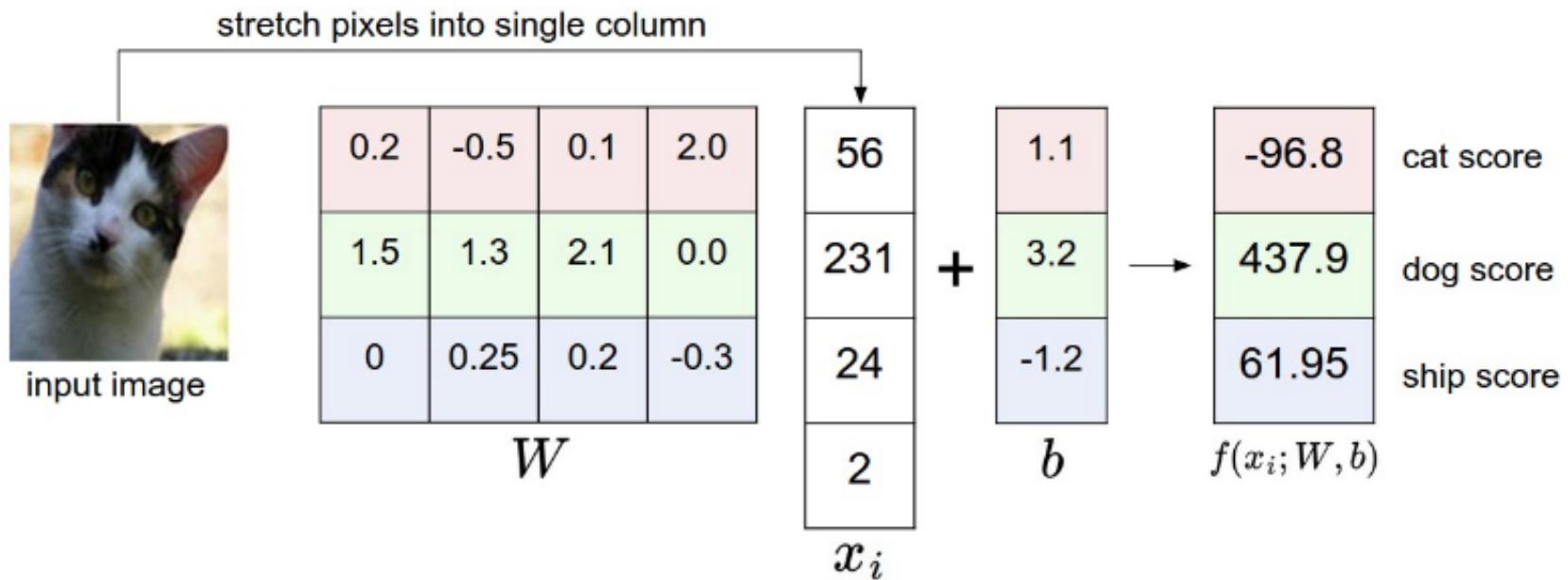
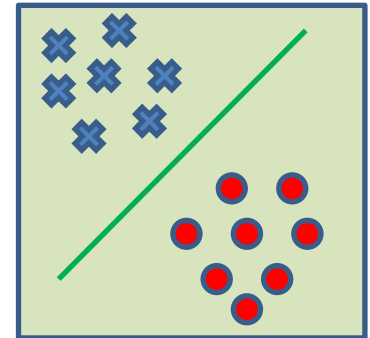
$$f(x_i, W, b) = Wx_i + b$$

- Use pairs of inputs and labels to find the **weights matrix** W and the **bias vector** b
 - The weights and biases are the **parameters** of the function f
- Several methods have been used to find the optimal set of parameters of a linear classifier
 - A common method of choice is the **Perceptron** algorithm, where the parameters are updated until a minimal error is reached (single layer, does not use backpropagation)
- Linear classifier is a simple approach, but it is a building block of advanced classification algorithms, such as SVM and neural networks
 - Earlier multi-layer neural networks were referred to as multi-layer perceptrons (MLPs)

Linear Classifier

Machine Learning Basics

- The **decision boundary** is linear
 - A straight line in 2D, a flat plane in 3D, a **hyperplane** in 3D and higher dimensional space
- Example: classify an input image
 - The selected parameters in this example are not good, because the predicted cat score is low



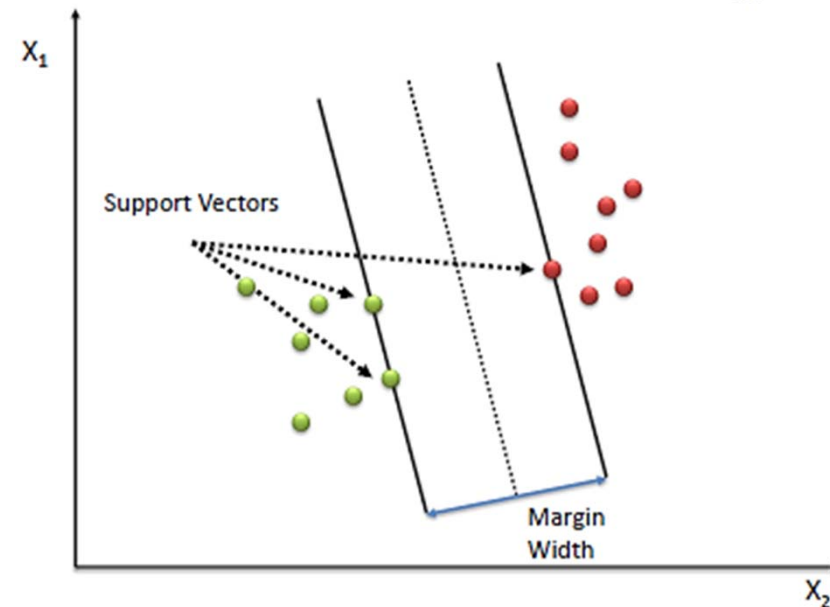
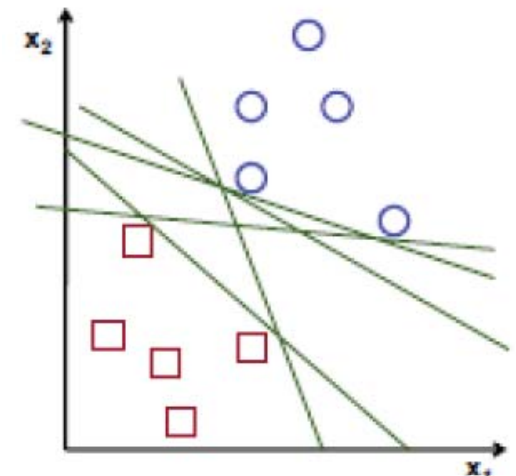
Support Vector Machines

Machine Learning Basics

- *Support vector machines (SVM)*

- How to find the best decision boundary?
 - All lines in the figure correctly separate the 2 classes
 - The line that is farthest from all training examples will have better generalization capabilities
- SVM solves an optimization problem:
 - First, identify a **decision boundary** that correctly classifies the examples
 - Next, increase the geometric margin between the boundary and all examples
- The data points that define the maximum margin width are called **support vectors**
- Find W and b by solving:

$$\min \frac{1}{2} \|w\|^2$$
$$s.t. y_i(w \cdot x_i + b) \geq 1, \forall x_i$$



Linear vs Non-linear Techniques

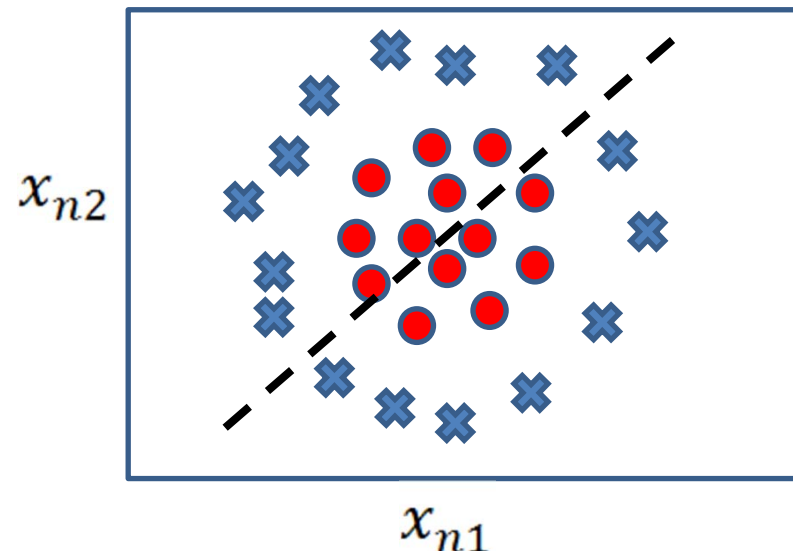
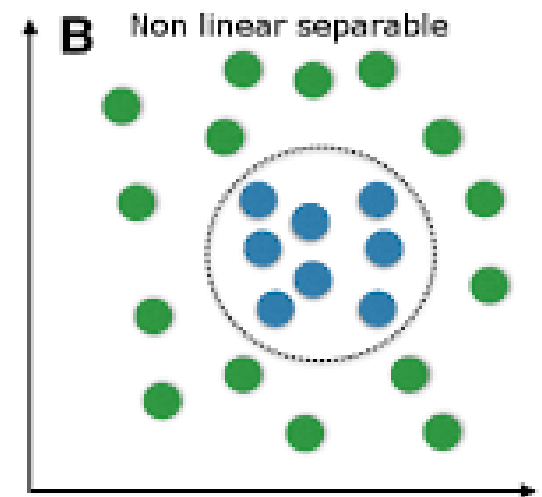
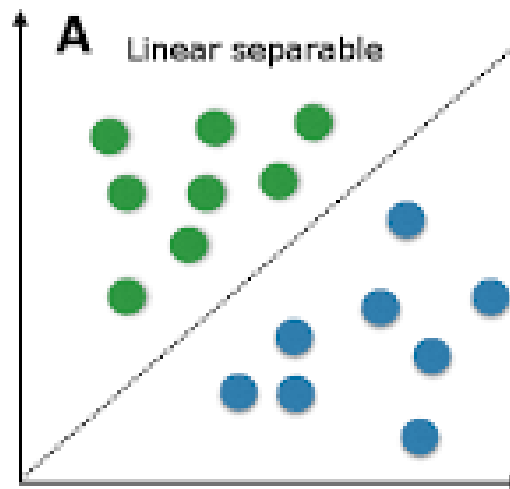
Linear vs Non-linear Techniques

- Linear classification techniques
 - Linear classifier
 - Perceptron
 - Logistic regression
 - Linear SVM
 - Naïve Bayes
- Non-linear classification techniques
 - k -nearest neighbors
 - Non-linear SVM
 - Neural networks
 - Decision trees
 - Random forest

Linear vs Non-linear Techniques

Linear vs Non-linear Techniques

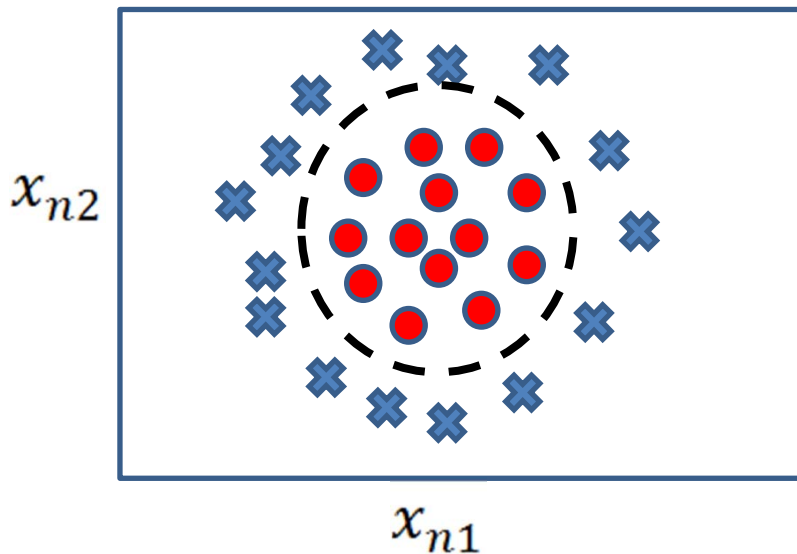
- For some tasks, input data can be linearly separable, and linear classifiers can be suitably applied
- For other tasks, linear classifiers may have difficulties to produce adequate decision boundaries



Non-linear Techniques

Linear vs Non-linear Techniques

- Non-linear classification
 - Features z_i are obtained as **non-linear functions** of the inputs x_i
 - It results in non-linear decision boundaries
 - Can deal with non-linearly separable data



Inputs: $x_i = [x_{n1} \quad x_{n2}]$



Features: $z_i = [x_{n1} \quad x_{n2} \quad x_{n1} \cdot x_{n2} \quad x_{n1}^2 \quad x_{n2}^2]$



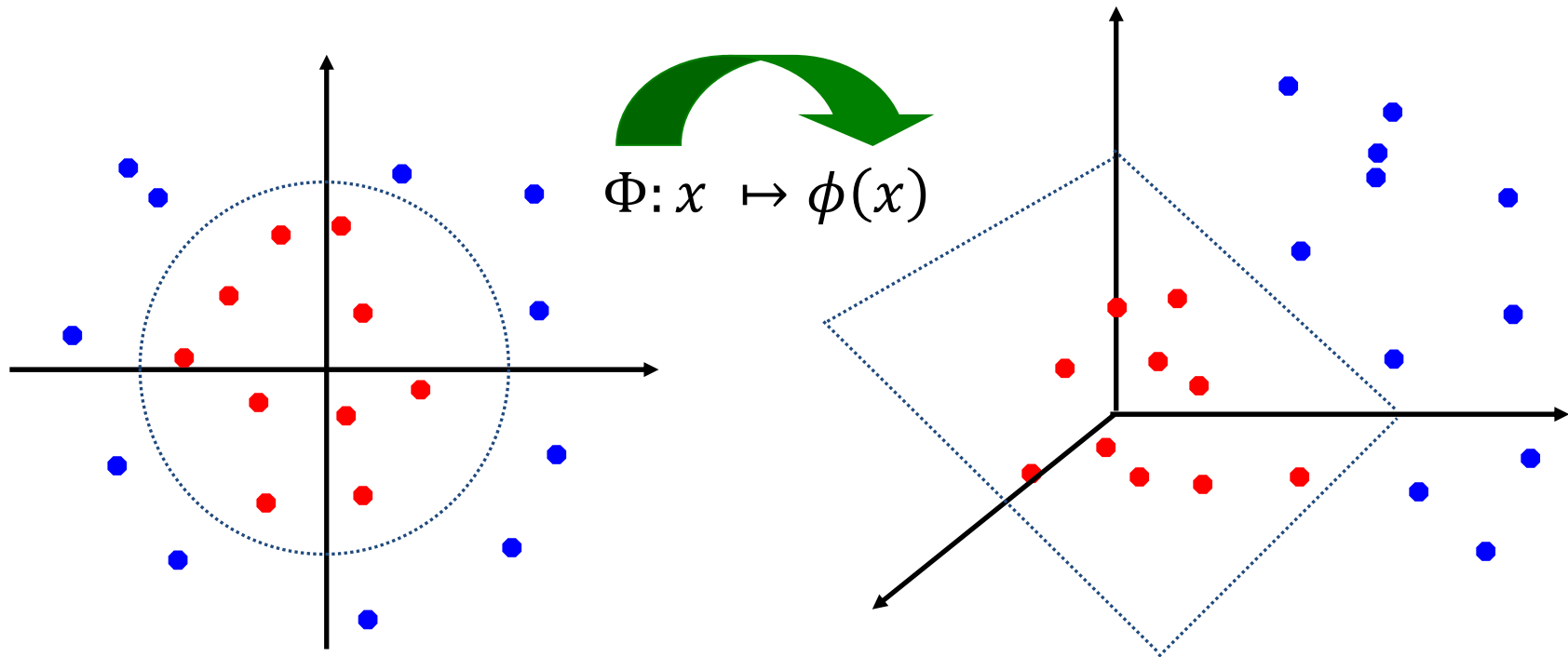
Outputs: $f(x_i, W, b) = Wz_i + b$

Non-linear Support Vector Machines

Linear vs Non-linear Techniques

- **Non-linear SVM**

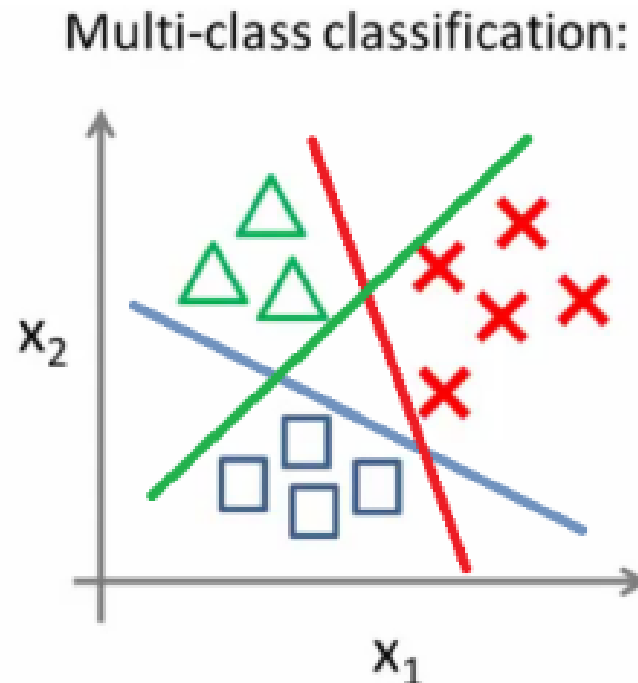
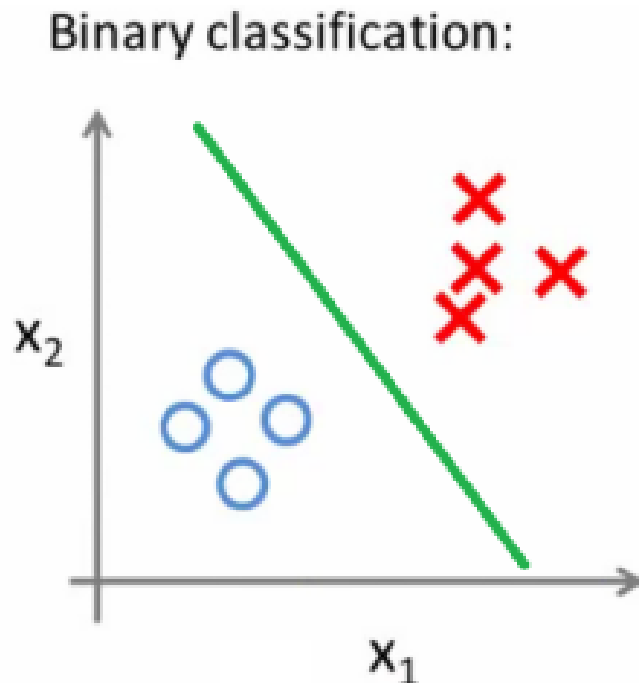
- The original input space is mapped to a higher-dimensional feature space where the training set is linearly separable
- Define a non-linear kernel function to calculate a non-linear decision boundary in the original feature space



Binary vs Multi-class Classification

Binary vs Multi-class Classification

- A classification problem with only 2 classes is referred to as *binary classification*
 - The output labels are 0 or 1
 - E.g., benign or malignant tumor, spam or no-spam email
- A problem with 3 or more classes is referred to as *multi-class classification*

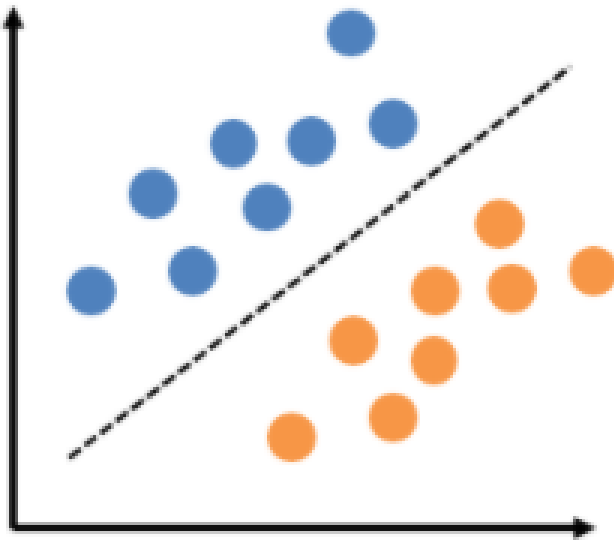


Binary vs Multi-class Classification

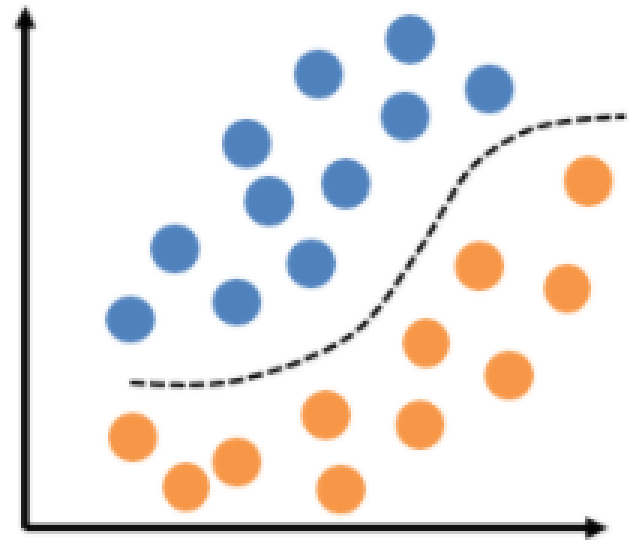
Binary vs Multi-class Classification

- Both the binary and multi-class classification problems can be linearly or non-linearly separated
 - Figure: linearly and non-linearly separated data for binary classification problem

Linear



Nonlinear



Computer Vision Tasks

Machine Learning Basics

- Computer vision has been the primary area of interest for ML
- The tasks include: classification, localization, object detection, instance segmentation

Classification



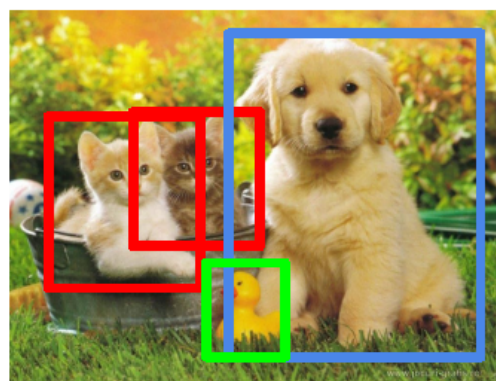
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



CAT, DOG, DUCK

Single object

Multiple objects

No-Free-Lunch Theorem

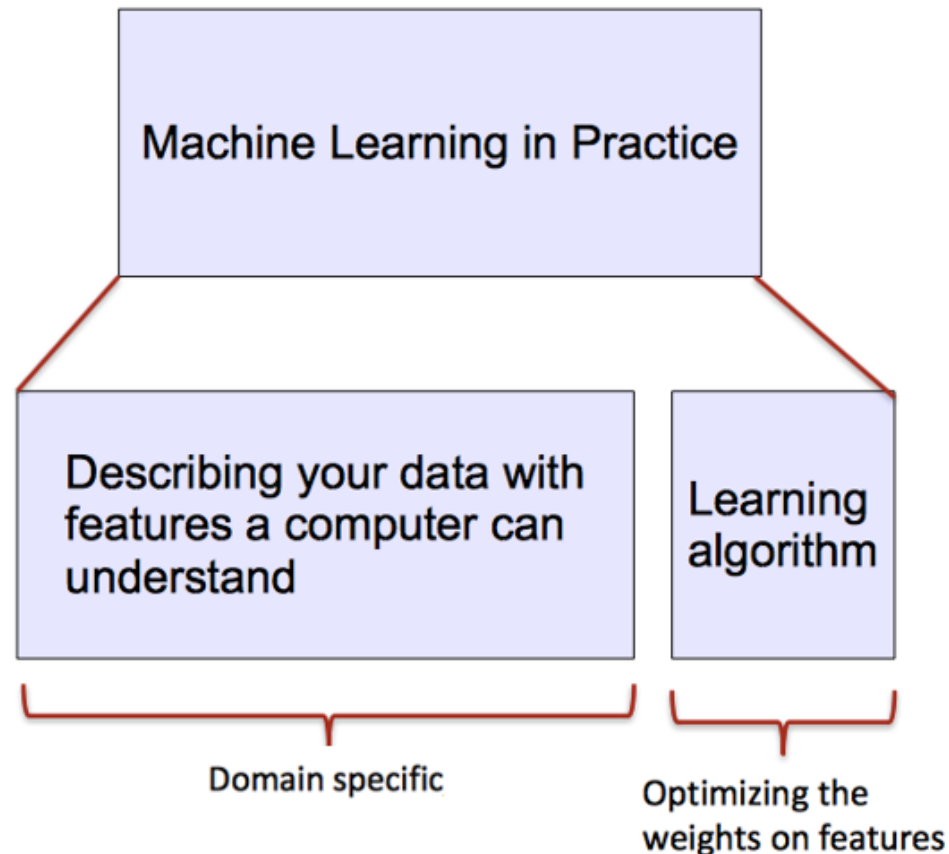
Machine Learning Basics

- [Wolpert \(2002\) - The Supervised Learning No-Free-Lunch Theorems](#)
- The derived classification models for supervised learning are simplifications of the reality
 - The simplifications are based on certain assumptions
 - The assumptions fail in some situations
 - E.g., due to inability to perfectly estimate ML model parameters from limited data
- In summary, *No-Free-Lunch Theorem* states:
 - **No single classifier works the best for all possible problems**
 - Since we need to make assumptions to generalize

ML vs. Deep Learning

Introduction to Deep Learning

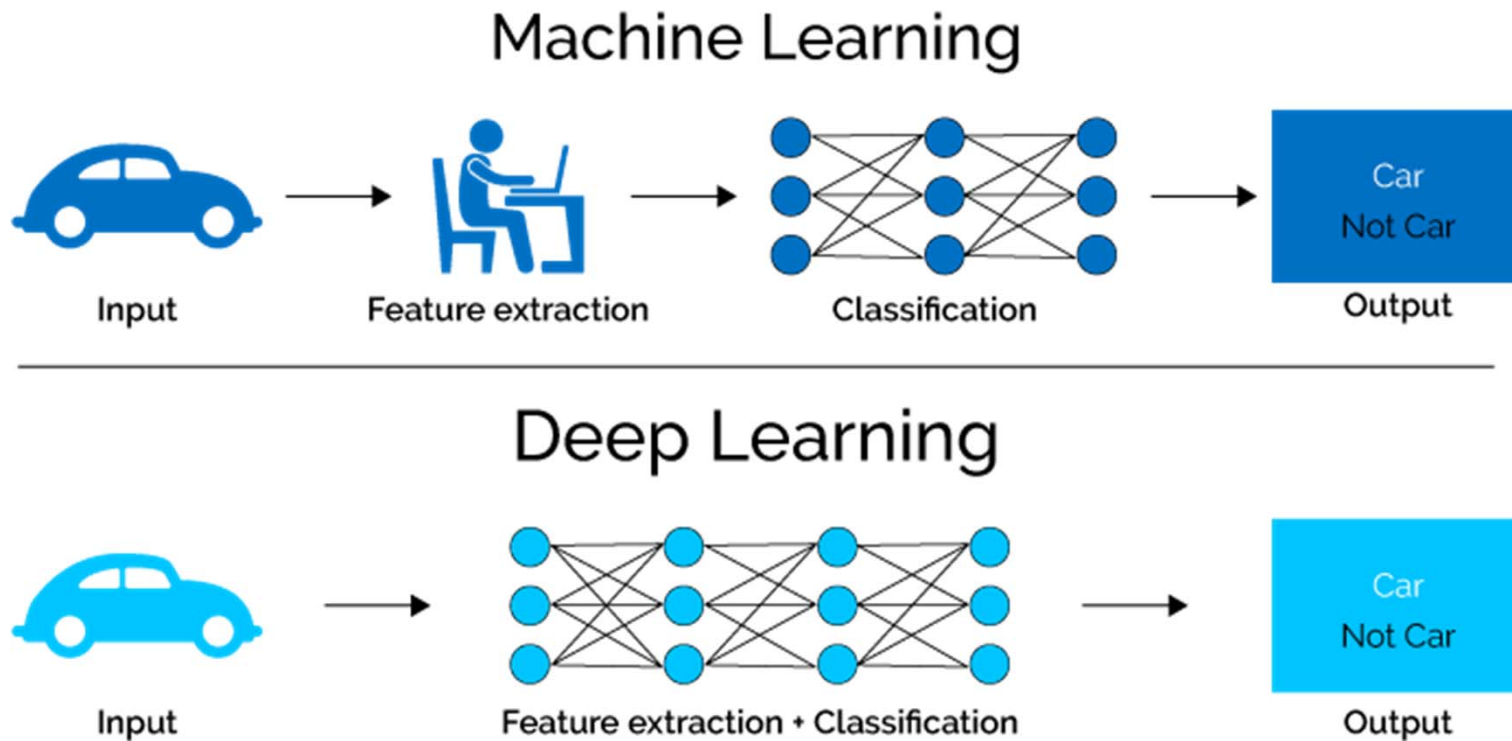
- Conventional machine learning methods rely on **human-designed feature representations**
 - ML becomes just optimizing weights to best make a final prediction



ML vs. Deep Learning

Introduction to Deep Learning

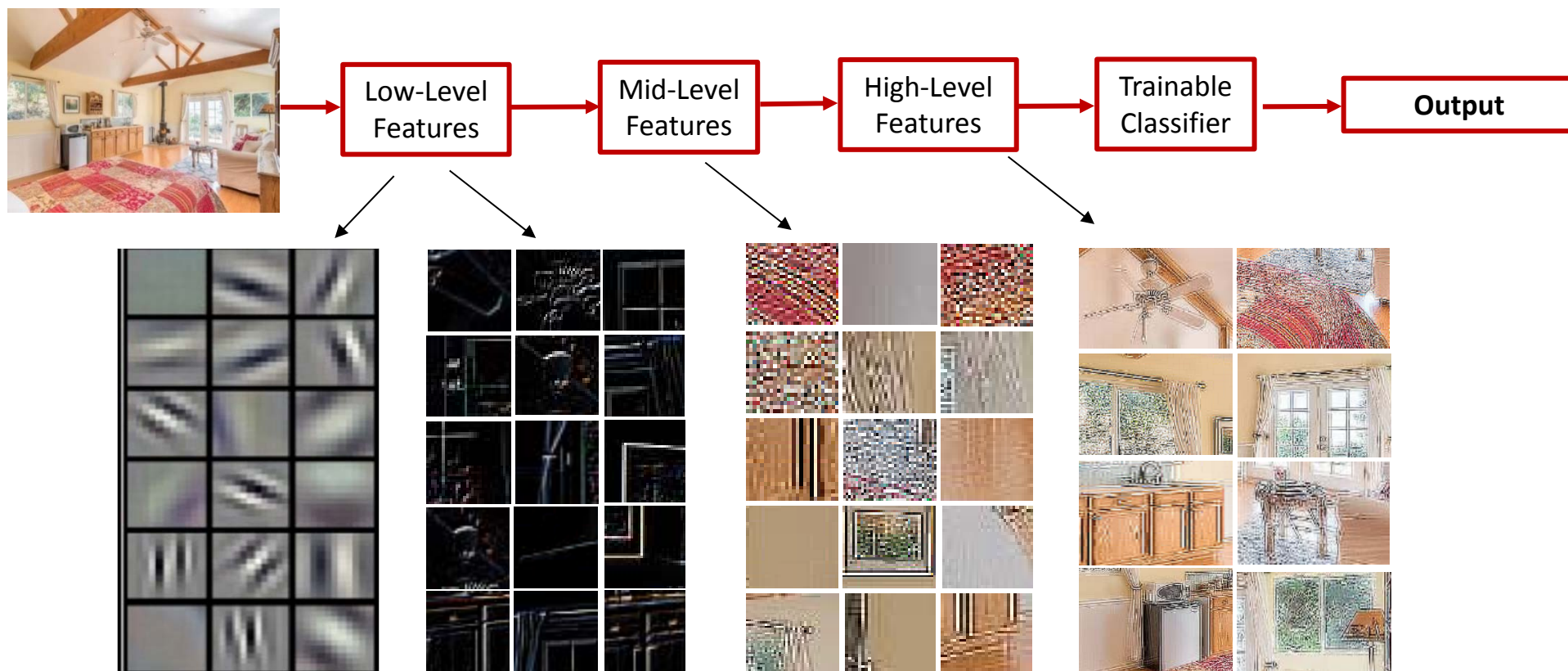
- **Deep learning** (DL) is a machine learning subfield that uses multiple layers for learning data representations
 - DL is exceptionally effective at learning patterns



ML vs. Deep Learning

Introduction to Deep Learning

- DL applies a multi-layer process for learning rich hierarchical features (i.e., data representations)
 - Input image pixels → Edges → Textures → Parts → Objects



Why is DL Useful?

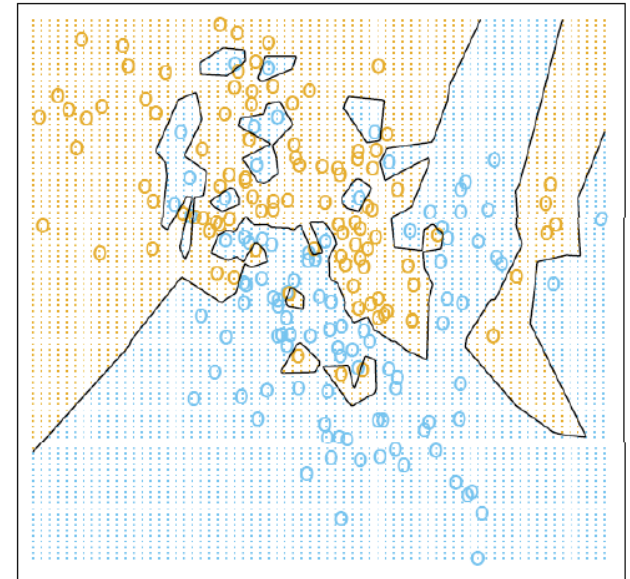
Introduction to Deep Learning

- DL provides a flexible, learnable framework for representing visual, text, linguistic information
 - Can learn in supervised and unsupervised manner
- DL represents an effective end-to-end learning system
- ~~Requires large amounts of training data~~ Not anymore with Transformers and Transfer Learning
- Since about 2010, DL has outperformed other ML techniques
 - First in vision and speech, then NLP, and other applications

Representational Power

Introduction to Deep Learning

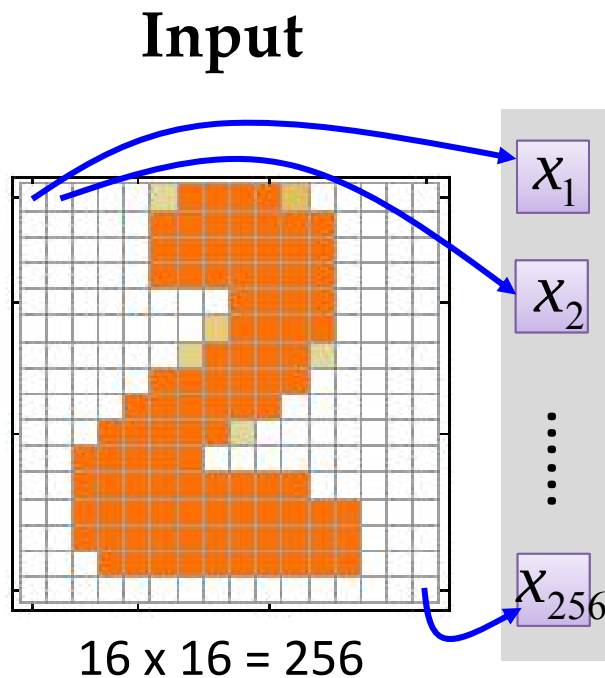
- NNs with at least one hidden layer are **universal approximators**
 - Given any continuous function $h(x)$ and some $\epsilon > 0$, there exists a NN with one hidden layer (and with a reasonable choice of non-linearity) described with the function $f(x)$, such that $\forall x, |h(x) - f(x)| < \epsilon$
 - I.e., NN can approximate any arbitrary complex continuous function
- NNs use nonlinear mapping of the inputs x to the outputs $f(x)$ to compute complex decision boundaries
- But then, why use deeper NNs?
 - The fact that deep NNs work better is an empirical observation
 - Mathematically, deep NNs have the same representational power as a one-layer NN



Introduction to Neural Networks

Introduction to Neural Networks

- Handwritten digit recognition (**MNIST dataset**)
 - The intensity of each pixel is considered an **input** element
 - **Output** is the class of the digit



Ink $\rightarrow$ 1
No ink $\rightarrow$ 0



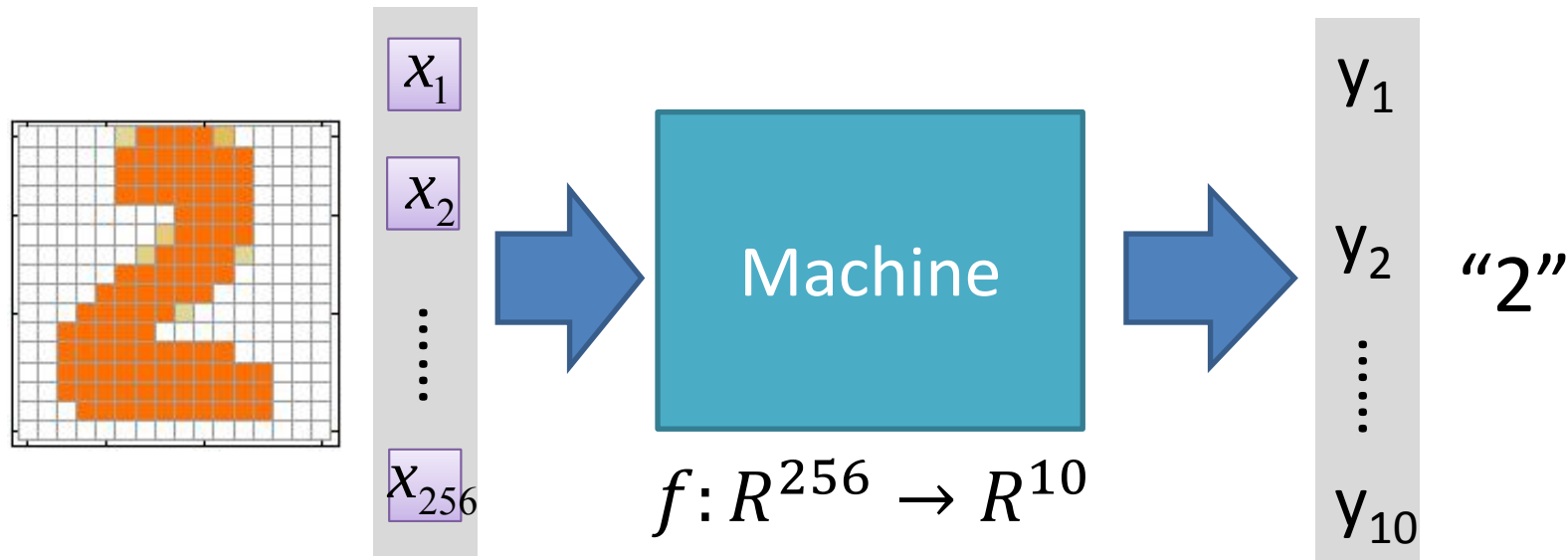
The image is "2"

Each dimension represents the confidence of a digit

Introduction to Neural Networks

Introduction to Neural Networks

- Handwritten digit recognition

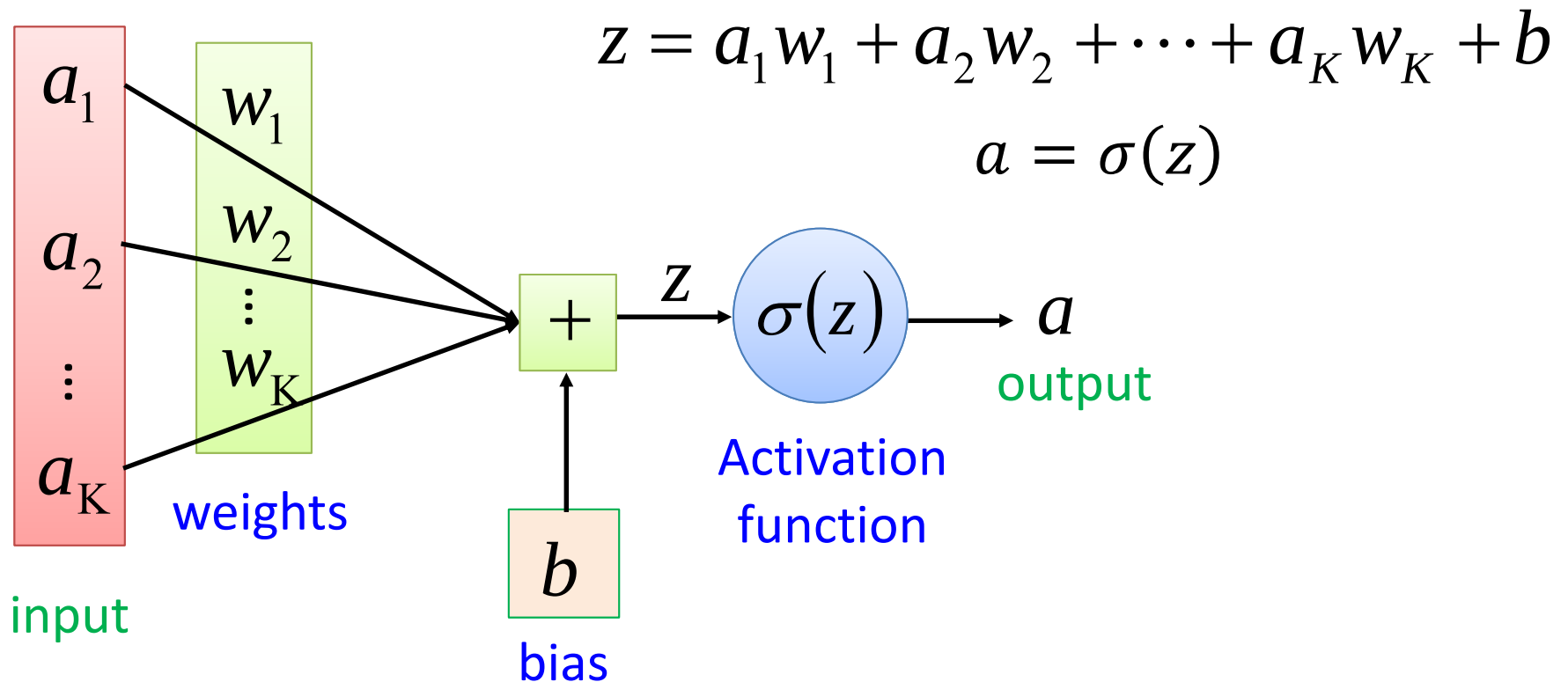


The function f is represented by a neural network

Elements of Neural Networks

Introduction to Neural Networks

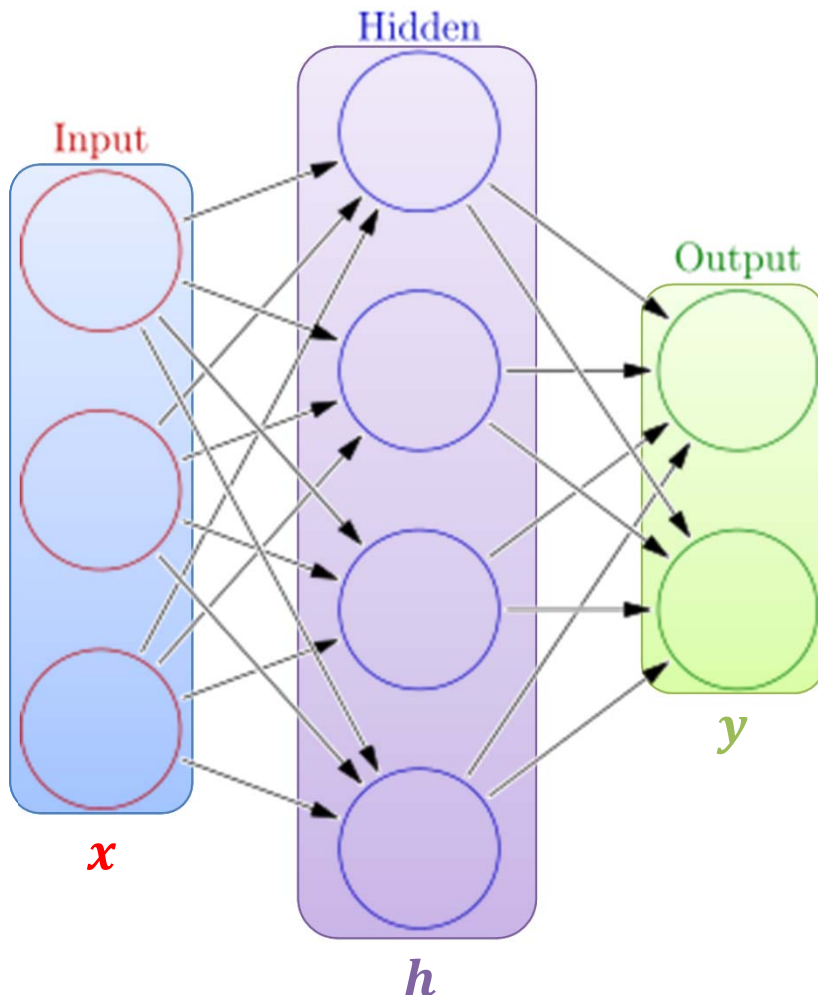
- NNs consist of hidden layers with neurons (i.e., computational units)
- A single **neuron** maps a set of inputs into an output number, or $f: R^K \rightarrow R$



Elements of Neural Networks

Introduction to Neural Networks

- A NN with one hidden layer and one output layer



Weights Biases

$$\text{hidden layer } h = \sigma(W_1 x + b_1)$$
$$\text{output layer } y = \sigma(W_2 h + b_2)$$

Activation functions

Diagrammatic description: The text 'Weights' and 'Biases' is at the top. Two blue arrows point from 'Weights' to the W_1 and W_2 terms in the equations. Two blue arrows point from 'Biases' to the b_1 and b_2 terms. Two blue arrows point from the σ (activation function) terms to the text 'Activation functions' below.

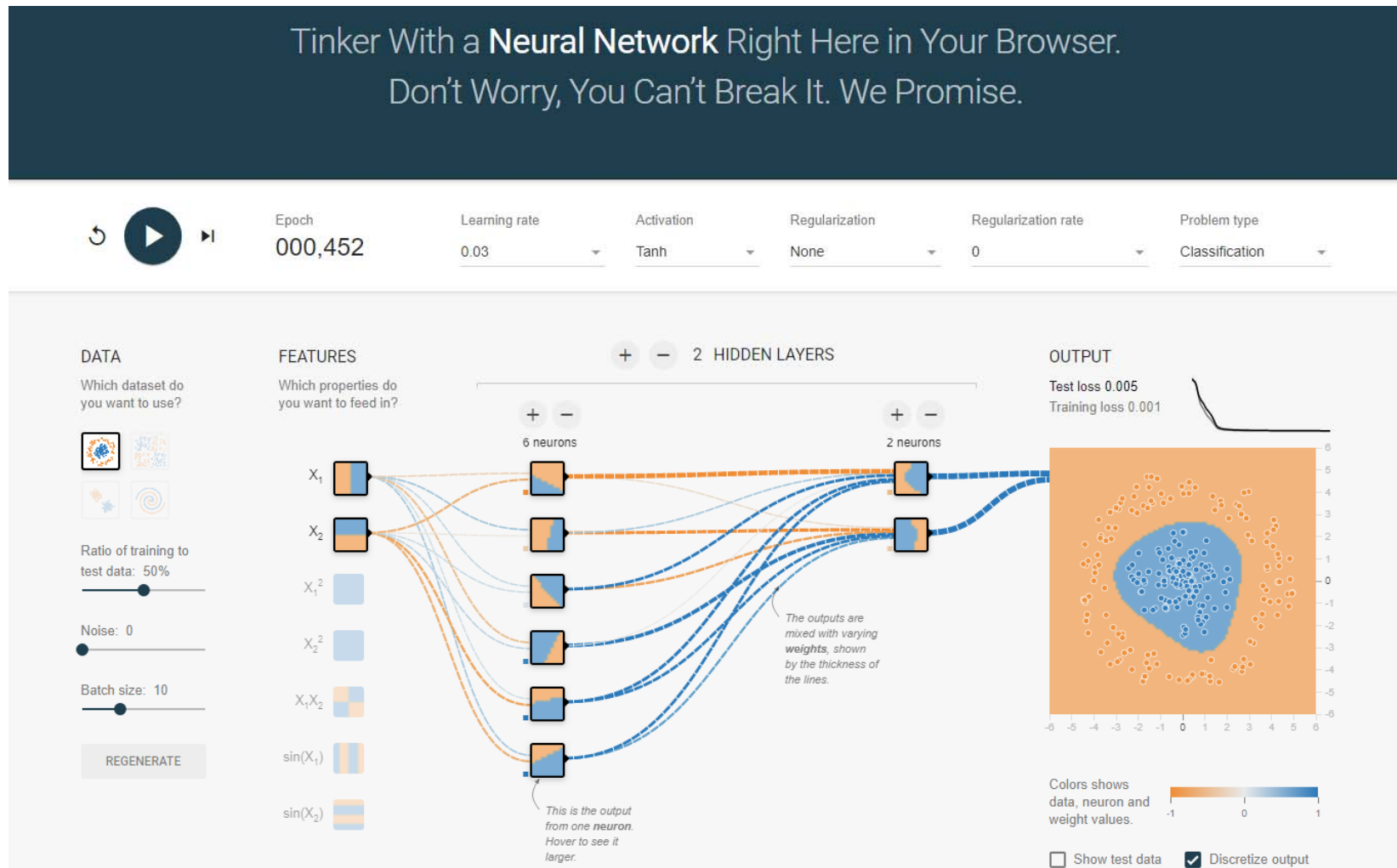
$$4 + 2 = 6 \text{ neurons (not counting inputs)}$$
$$[3 \times 4] + [4 \times 2] = 20 \text{ weights}$$
$$4 + 2 = 6 \text{ biases}$$

$$26 \text{ learnable parameters}$$

Elements of Neural Networks

Introduction to Neural Networks

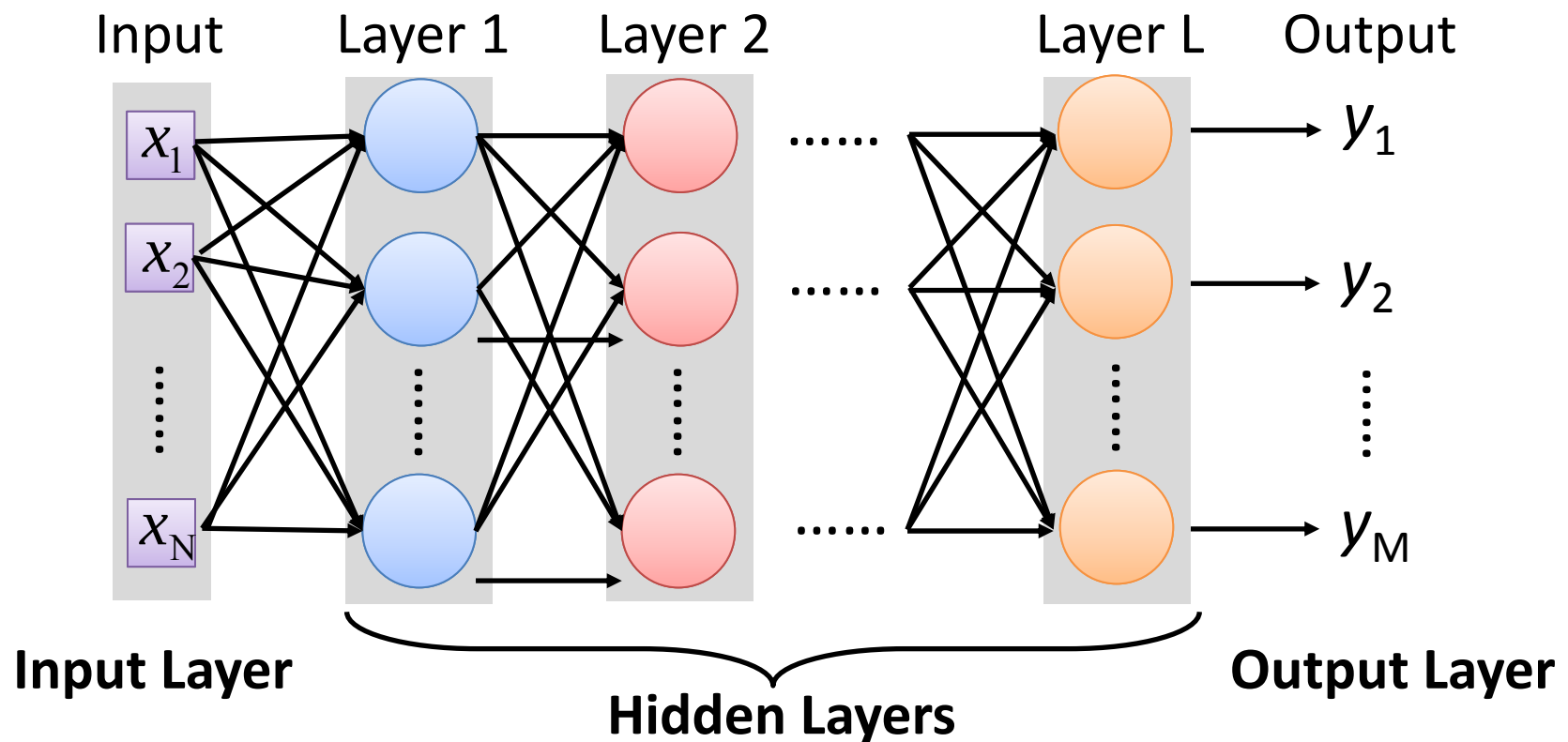
- A neural network playground [link](#)



Elements of Neural Networks

Introduction to Neural Networks

- Deep NNs have many hidden layers
 - **Fully-connected** (**dense**) layers (a.k.a. **Multi-Layer Perceptron** or MLP)
 - Each neuron is connected to all neurons in the succeeding layer

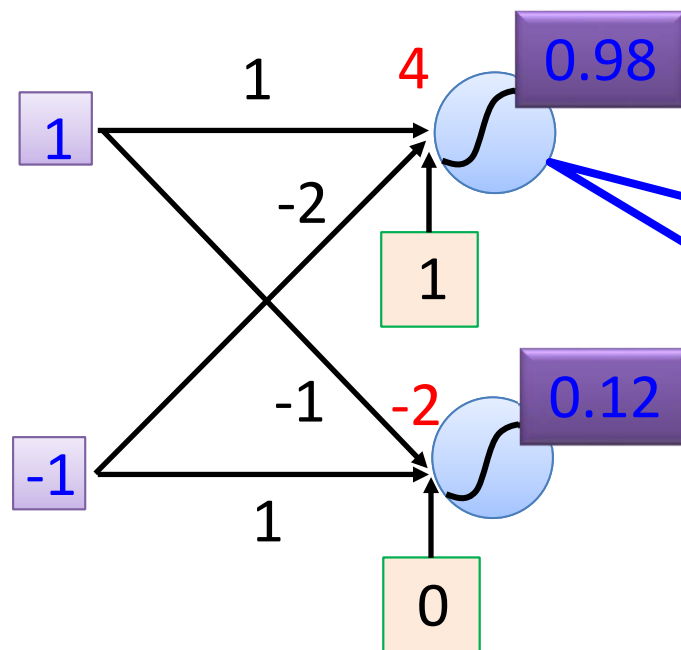


Elements of Neural Networks

Introduction to Neural Networks

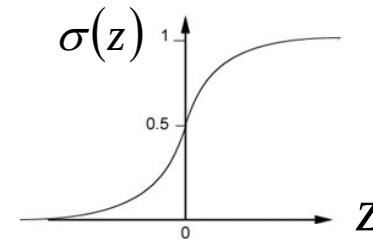
- A simple network, toy example

$$(1 \cdot 1) + (-1) \cdot (-2) + 1 = 4$$



Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

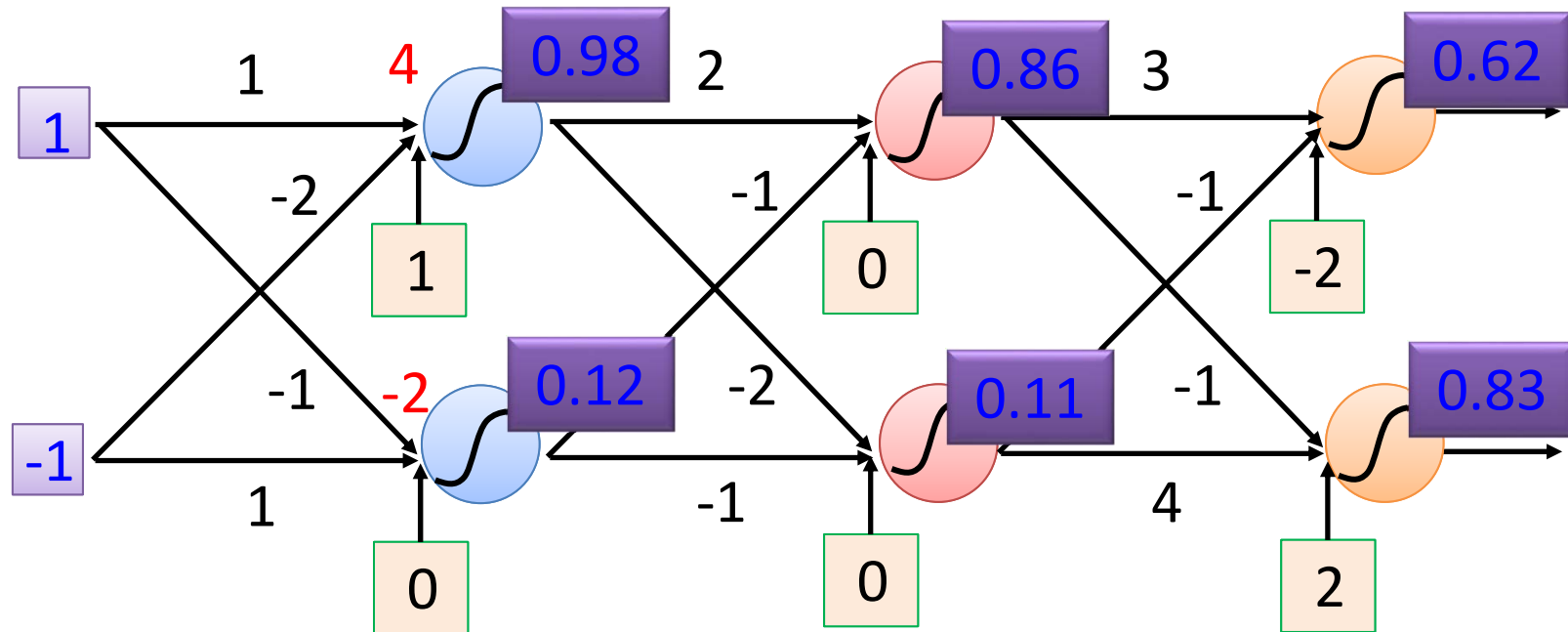


$$1 \cdot (-1) + (-1) \cdot 1 + 0 = -2$$

Elements of Neural Networks

Introduction to Neural Networks

- A simple network, toy example (cont'd)
 - For an input vector $[1 \ -1]^T$, the output is $[0.62 \ 0.83]^T$

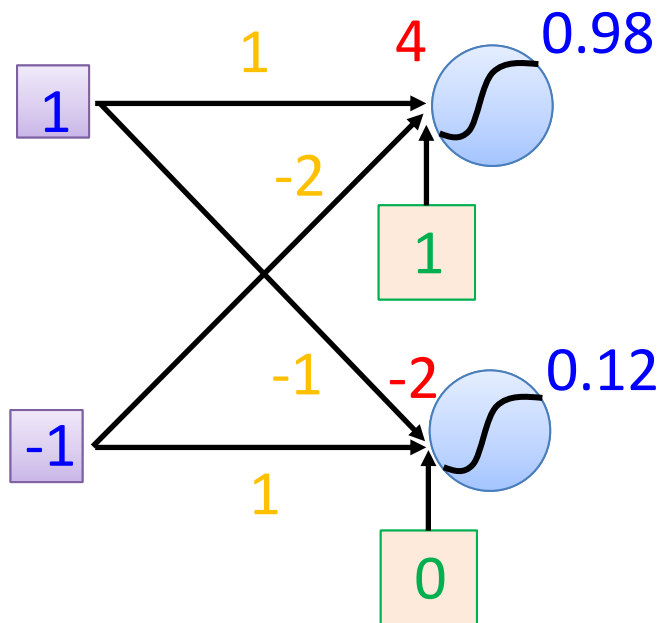


$$f: R^2 \rightarrow R^2 \quad f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$$

Matrix Operation

Introduction to Neural Networks

- Matrix operations are helpful when working with multidimensional inputs and outputs



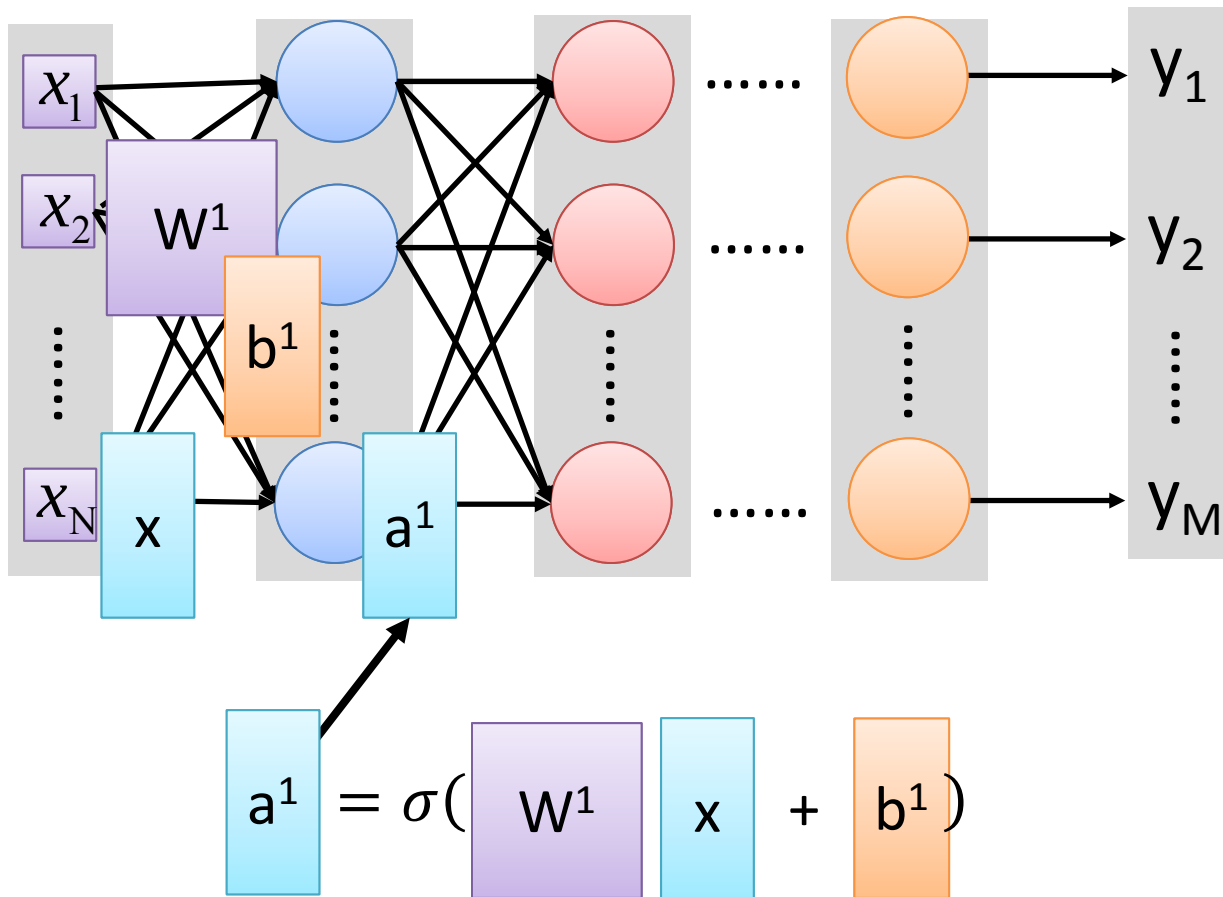
$$\sigma(W x + b) = a$$

$$\sigma(\underbrace{\begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\begin{bmatrix} 4 \\ -2 \end{bmatrix}}) = \begin{bmatrix} 0.98 \\ 0.12 \end{bmatrix}$$

Matrix Operation

Introduction to Neural Networks

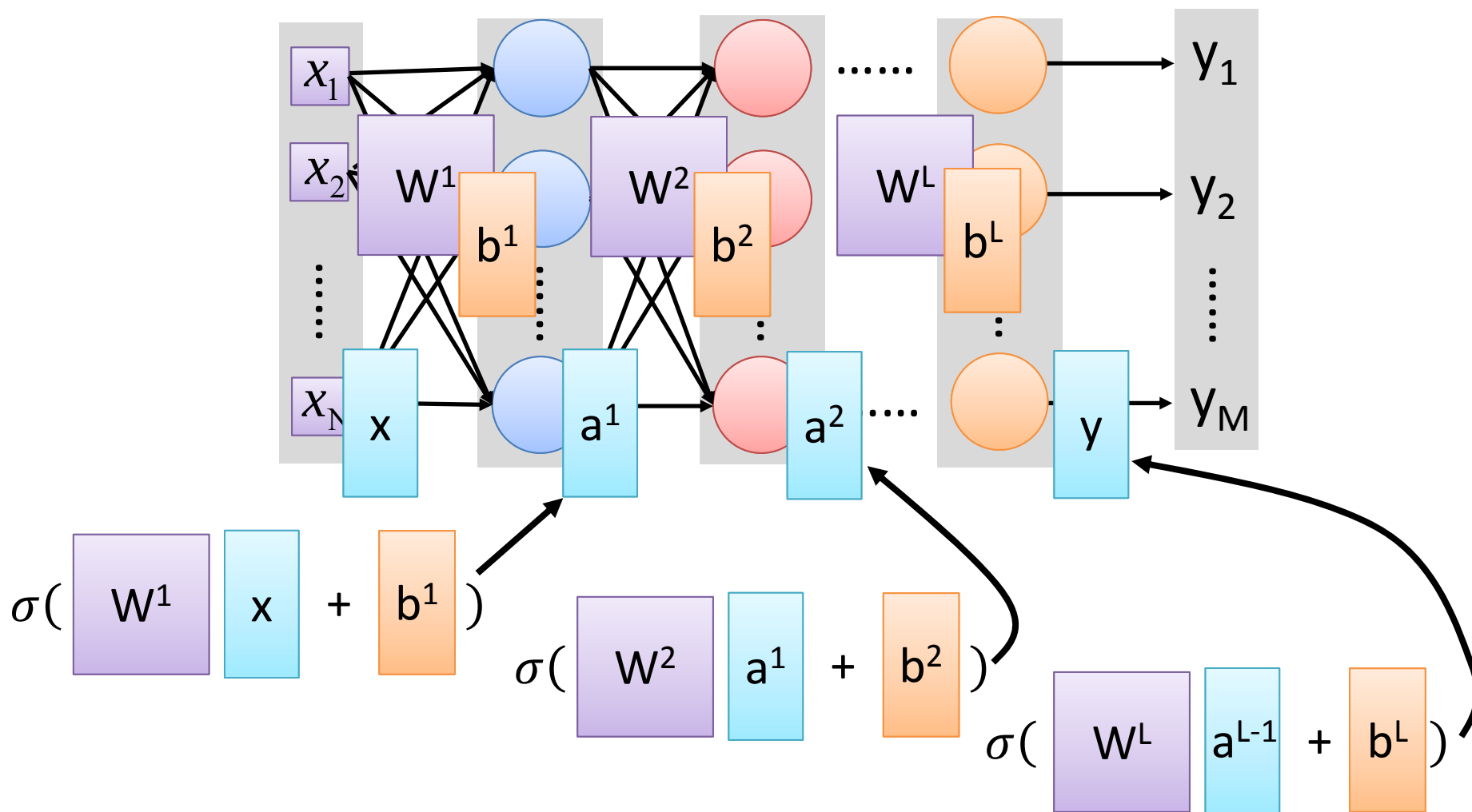
- Multilayer NN, matrix calculations for the first layer
 - Input vector x , weights matrix W^1 , bias vector b^1 , output vector a^1



Matrix Operation

Introduction to Neural Networks

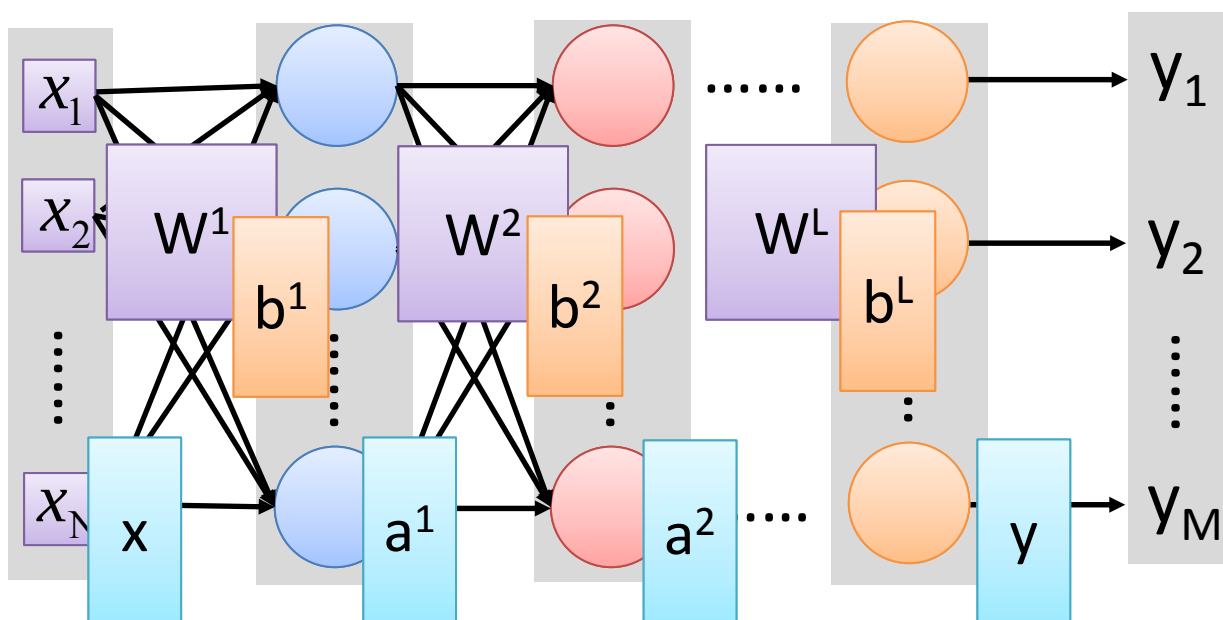
- Multilayer NN, matrix calculations for all layers



Matrix Operation

Introduction to Neural Networks

- Multilayer NN, function f maps inputs x to outputs y , i.e., $y = f(x)$



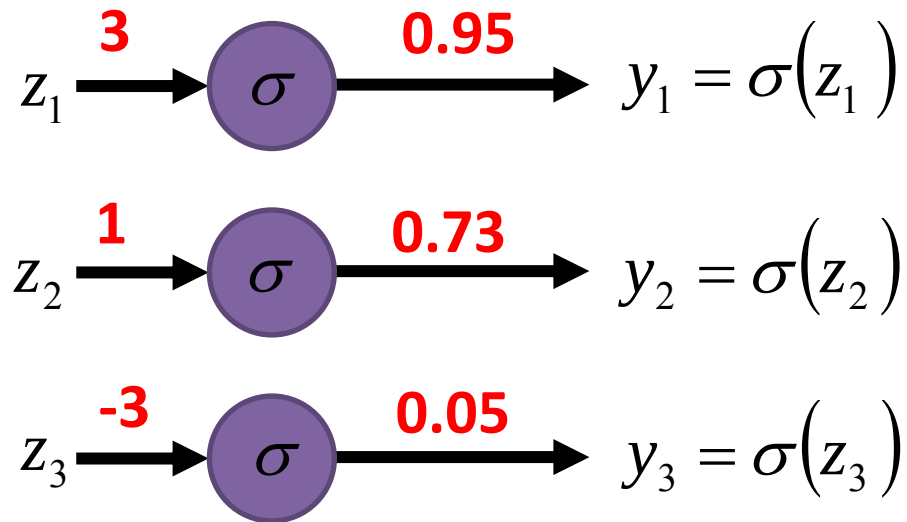
$$y = f(x) = \sigma(W^L \dots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \dots + b^L)$$

Softmax Layer

Introduction to Neural Networks

- In **multi-class classification** tasks, the output layer is typically a *softmax layer*
 - I.e., it employs a *softmax activation function*
 - If a layer with a sigmoid activation function is used as the output layer instead, the predictions by the NN may not be easy to interpret
 - Note that an output layer with sigmoid activations can still be used for binary classification

A Layer with Sigmoid Activations



Softmax Layer

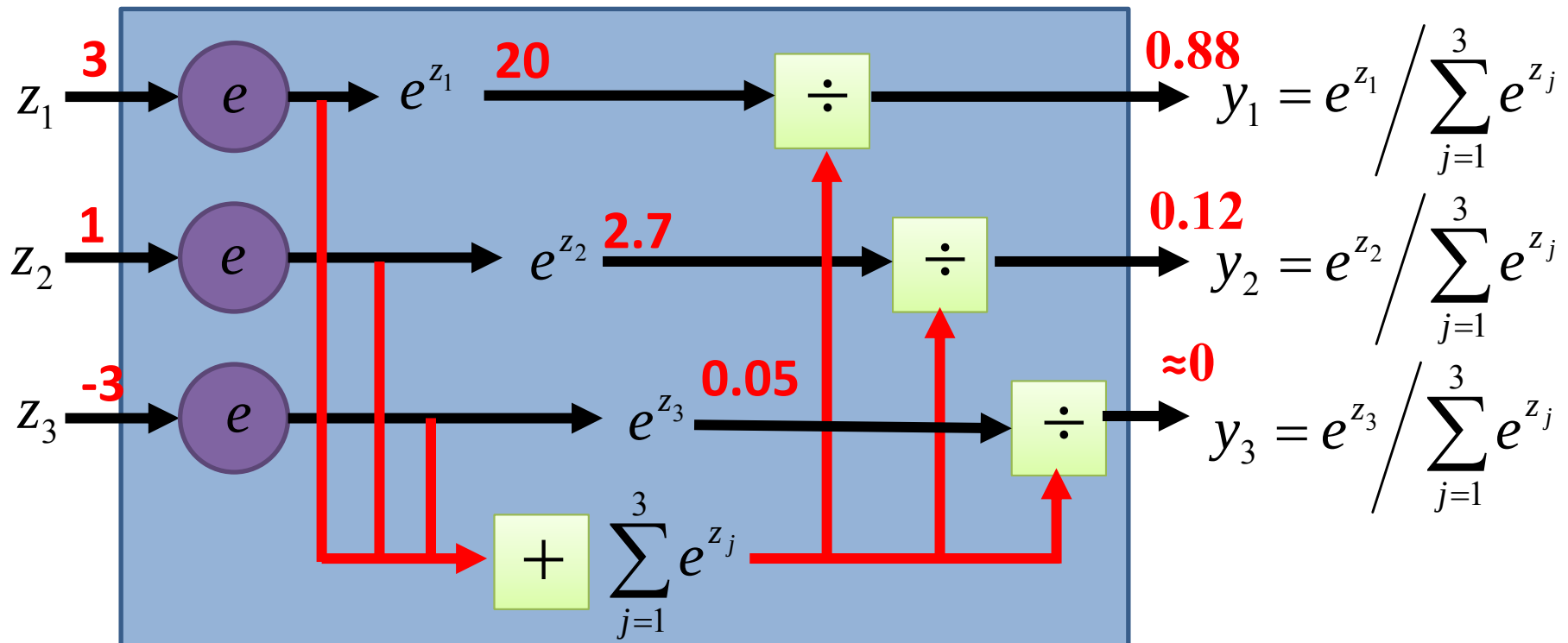
Introduction to Neural Networks

- The **softmax layer** applies softmax activations to output a probability value in the range $[0, 1]$
 - The values z inputted to the softmax layer are referred to as *logits*

Probability:

- $0 < y_i < 1$
- $\sum_i y_i = 1$

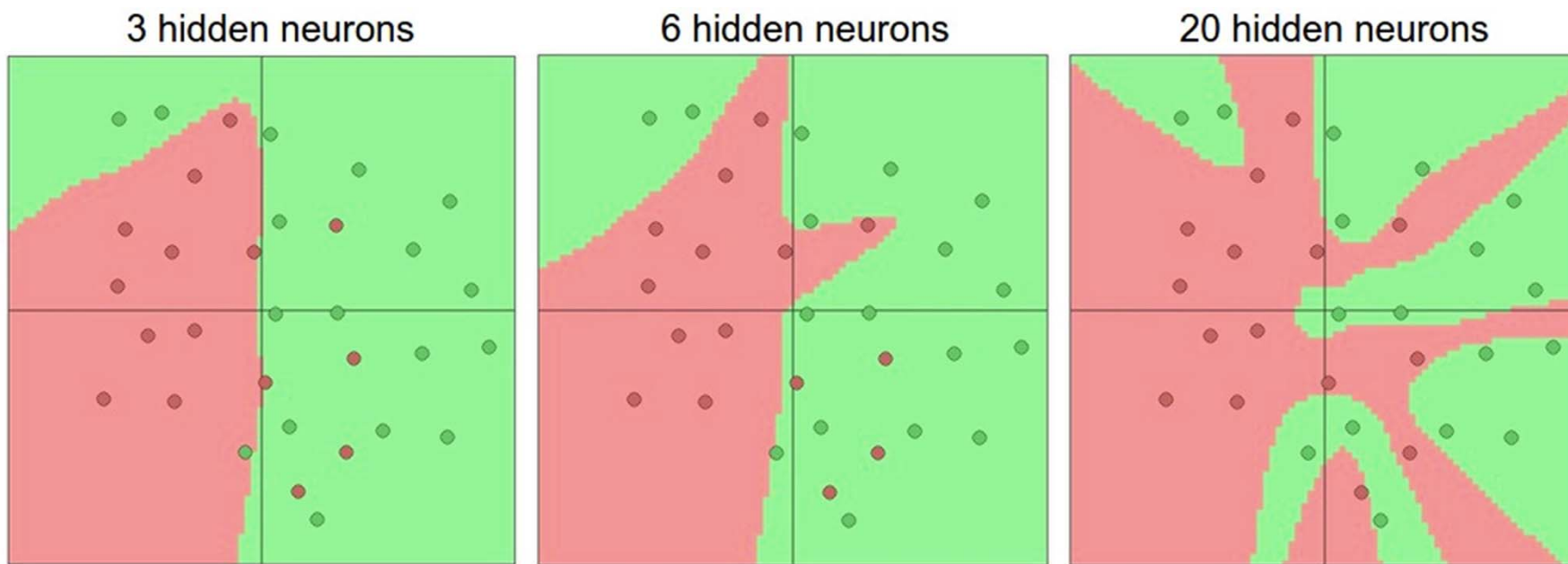
A Softmax Layer



Activation Functions

Introduction to Neural Networks

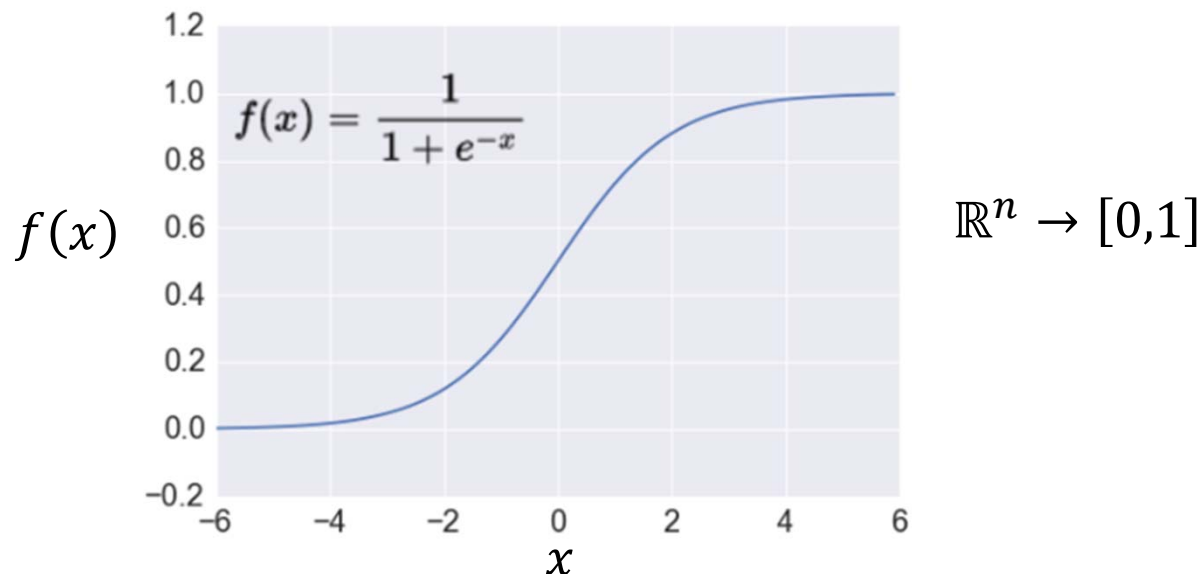
- **Non-linear activations** are needed to learn complex (non-linear) data representations
 - Otherwise, NNs would be just a linear function (such as $W_1 W_2 x = Wx$)
 - NNs with large number of layers (and neurons) can approximate more complex functions
 - Figure: more neurons improve representation (but, may overfit)



Activation: Sigmoid

Introduction to Neural Networks

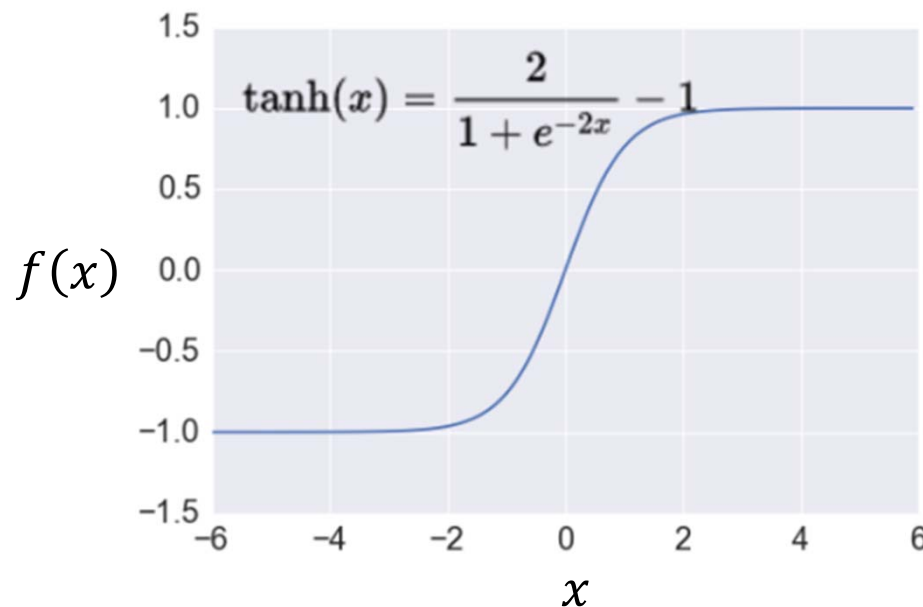
- **Sigmoid function** σ : takes a real-valued number and “squashes” it into the range between 0 and 1
 - The output can be interpreted as the firing rate of a biological neuron
 - Not firing = 0; Fully firing = 1
 - When the neuron’s activation are 0 or 1, sigmoid neurons saturate
 - Gradients at these regions are almost zero (almost no signal will flow)
 - Sigmoid activations are less common in modern NNs



Activation: Tanh

Introduction to Neural Networks

- **Tanh function**: takes a real-valued number and “squashes” it into range between -1 and 1
 - Like sigmoid, tanh neurons saturate
 - Unlike sigmoid, the output is zero-centered
 - It is therefore preferred than sigmoid
 - Tanh is a scaled sigmoid: $\tanh(x) = 2 \cdot \sigma(2x) - 1$



$$\mathbb{R}^n \rightarrow [-1,1]$$

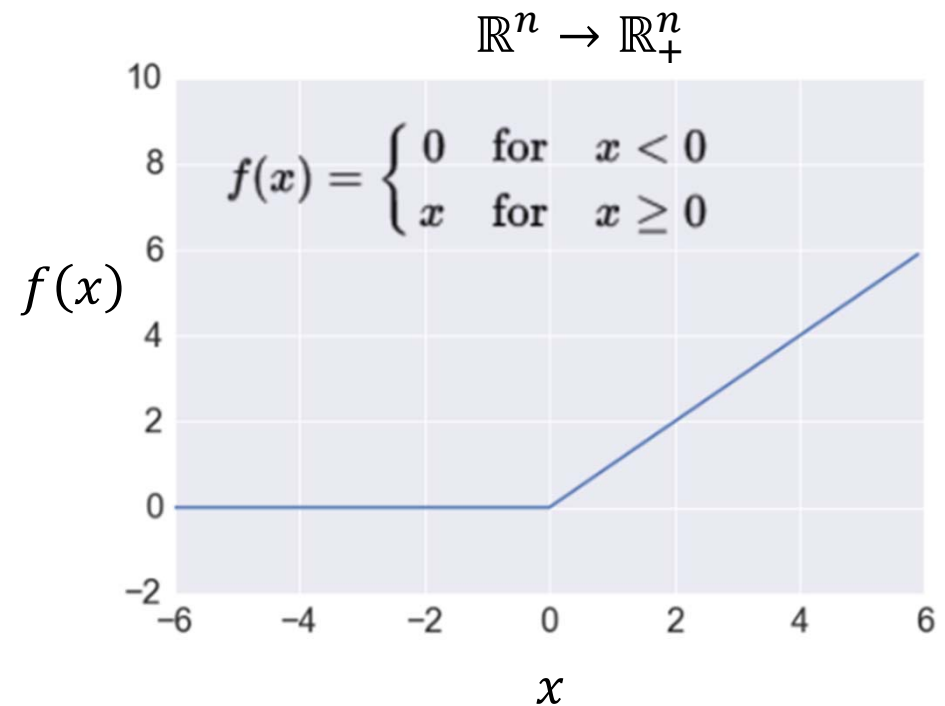
Activation: ReLU

Introduction to Neural Networks

- **ReLU** (Rectified Linear Unit): takes a real-valued number and thresholds it at zero

$$f(x) = \max(0, x)$$

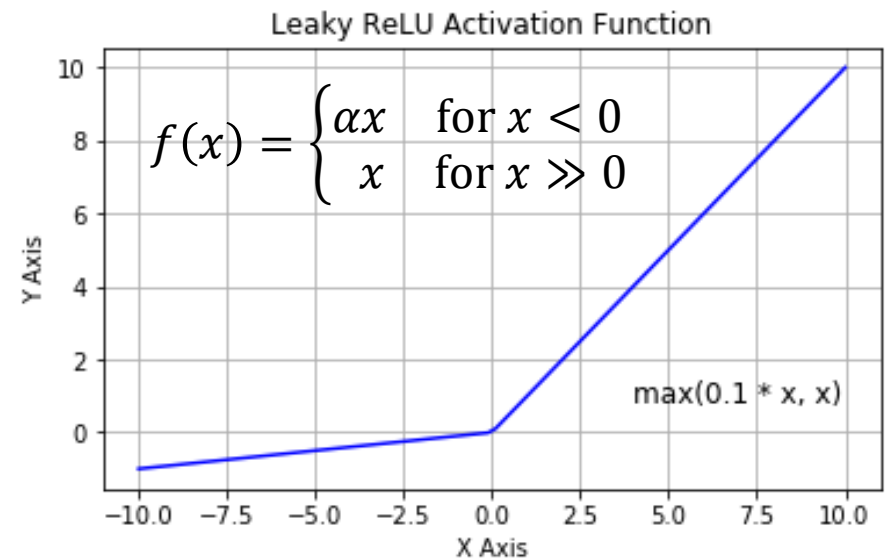
- Most modern deep NNs use ReLU activations
- ReLU is fast to compute
 - Compared to sigmoid, tanh
 - Simply threshold a matrix at zero
- Accelerates the convergence of gradient descent
 - Due to linear, non-saturating form
- Prevents the gradient vanishing problem



Activation: Leaky ReLU

Introduction to Neural Networks

- The problem of ReLU activations: they can “die”
 - ReLU could cause weights to update in a way that the gradients can become zero and the neuron will not activate again on any data
 - E.g., when a large learning rate is used
- **Leaky ReLU** activation function is a variant of ReLU
 - Instead of the function being 0 when $x < 0$, a leaky ReLU has a small negative slope (e.g., $\alpha = 0.01$, or similar)
 - This resolves the dying ReLU problem
 - Most current works still use ReLU
 - With a proper setting of the learning rate, the problem of dying ReLU can be avoided



Activation: Linear Function

Introduction to Neural Networks

- **Linear function** means that the output signal is proportional to the input signal to the neuron
 - If the value of the constant c is 1, it is also called **identity activation function**
 - This activation type is used in regression problems
 - E.g., the last layer can have linear activation function, in order to output a real number (and not a class membership)

$$\mathbb{R}^n \rightarrow \mathbb{R}^n$$

